Cornell Bowers CIS
Computer Science

Applied High-Performance and Parallel Computing

Locality and Simulations (Vol. 2)

Professor Giulia Guidi

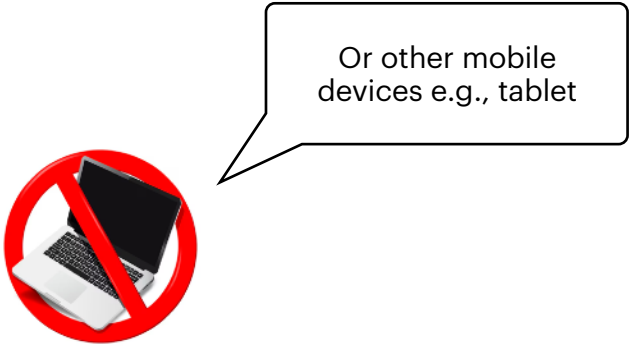
02/05/2026

A number of slides were taken from or inspired by slides by David Bindel, Jim Demmel, and Yoav Etsion

Plan for Today

- Locality and Simulation: Particles Simulation
- Locality and Simulation: ODEs and PDEs

no smartphones



no laptop

Why? Research shows that they slow us down in a learning environment.
I recommend **paper and pen** for taking notes

Credit to: https://users.cs.utah.edu/~pandey/courses/cs6530/fall23/index.html

Basic Types of Simulation

- Discrete event systems
  - *"Game of Life," Manufacturing systems, Finance, Circuits, Pac-man, etc.*

Today

- Particle systems:
  - *Billiard balls, Galaxies, Atoms, Circuits, Pinball, etc.*
- Lumped variables depending on continuous parameters
  - aka Ordinary Differential Equations (ODEs)
  - *Structural mechanics, Chemical kinetics, Circuits*
- Continuous variables depending on continuous parameters
  - aka Partial Differential Equations (PDEs)
  - *Heat, Elasticity, Electrostatics, Finance, Circuits, Medical Image Analysis*

Particle Systems

- **A particle system has:**
  - A finite number of particles
  - The particles move in space according to Newton's Laws (i.e. F = ma)
  - Time and position are continuous
- **Examples:**
  - The stars in space with laws of gravity
  - An electron beam in semiconductor manufacturing
  - The atoms in a molecule with electrostatic forces
  - The neutrons in a fission reactor
  - Balls in a pinball game

Forces in Particle Systems

- **Force on each particle can be subdivided:**

force = external_force + nearby_force + far_field_force

- **External force:**
  - Ocean current in sharks and fish world (S&F 1)
  - Externally imposed electric field in electron beam

- **Nearby force:**
  - Sharks attracted to eat nearby fish (S&F 5)
  - Balls on a billiard table bounce off of each other
  - Van der Waals forces in fluid (Gecko feet)

- **Far-field force:**
  - Fish attract other fish by gravity-like force (S&F 2)
  - Gravity, electrostatics

Parallelism in External Forces

- These are **the simplest**
- The force on each particle is **independent**
- Called **"embarrassingly parallel":**
  - Sometimes called "map reduce" by analogy

Parallelization:

- Evenly distribute particles on processors**:**
  - Any distribution if fine!
  - Because **locality is not an issue**
- For each particle on processor, apply the external force**:**
  - Also called "map" (e.g., absolute value)
  - You may need to "reduce" (e.g., compute maximum) to compute time step, other data

Parallelism in Nearby Forces

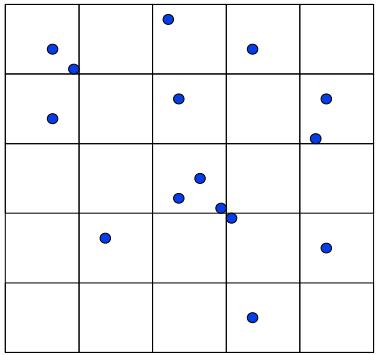
- Nearby forces require interaction and therefore **communication**
- Forces may depend on other nearby particles:
  - E.g., collisions
  - The simplest algorithm is $O(n^2)$: look at all pairs of particles to see if they collide

HW2 starter code is going to be **$O(n^2)$**

<u>Parallelization Strategy</u>:

- **Domain decomposition** based on where particles are located
  - $O(n/p)$ particles per processor (p) if evenly distributed
  - Processors don't necessarily need to cover the same domain area

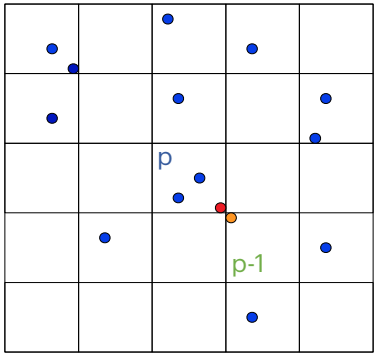
HW2 (Vol. 1) is make it **O(n)** using domain decomposition



Parallelism in Nearby Forces

Challenge #1: particle n_1 (red) on $processor_p$ has to interact with n_{45} (orange) on $processor_{p-1}$

Any idea how to deal with this?



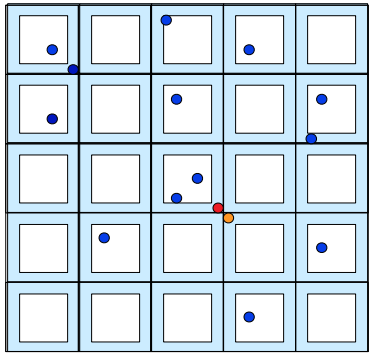
Parallelism in Nearby Forces

Challenge #1: particle n_1 (red) on processor_p has to interact with n_{45} (orange) on processor_{p-1}

Solution : communicate particles *near* boundary to neighboring processor(s)

- Region near boundary called **"ghost zone"** or **"halo"**

Programming pitfall in HW3 [MPI]: need to worry about not duplicating particles



Parallelism in Nearby Forces

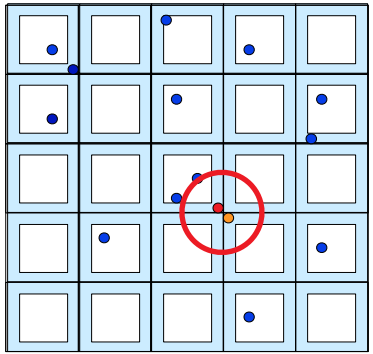
Challenge #1: particle n_1 (red) on $processor_p$ has to interact with n_{45} (orange) on $processor_{p-1}$

Solution : communicate particles *near* boundary to neighboring processor(s)

- Region near boundary called **"ghost zone"** or **"halo"**

Solution (continued): *low surface to volume ratio means low communication*

- Use ***squares***, not slabs, to *minimize* ghost zone sizes



You need to check for collisions between regions

Parallelism in Nearby Forces

Challenge #2: load imbalance, if particles cluster

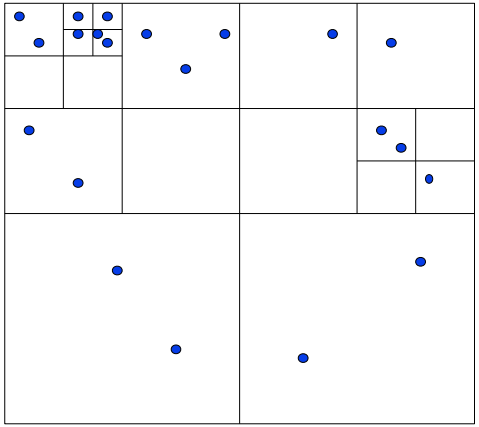
- Galaxies, electrons hitting a device wall

This doesn't come up in **HW2**, but may come up in real life!

Solution: to reduce load imbalance, divide space unevenly

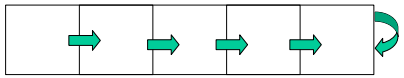
- Each region contains *roughly* equal number of particles
- Quad-tree in 2D, oct-tree in 3D
- You may *need to rebalance as particles move*, hopefully seldom

E.g., each square contains at most 3 particles



Parallelism in Far-Field Forces

- Far-field forces **involve all-to-all interaction** and therefore **communication**
- Force depends on all other particles:
  - E.g., gravity, protein folding
  - The simplest algorithm is $O(n^2)$ as in S&F 2, 4, 5.
  - Just decomposing space doesn't help since every particle must "visit" every other particle



One option is to implement it by rotating particles to processes in a round robin fashion:

- Use more **clever** algorithm to **reduce communication**
- Use more **clever** algorithm to **beat $O(n^2)$**

Far-Field Forces: Particle-Mesh Method

- Based on approximation
  - It superimposes a (mathematical) regular mesh
  - And, **"move"** particles to nearest grid point

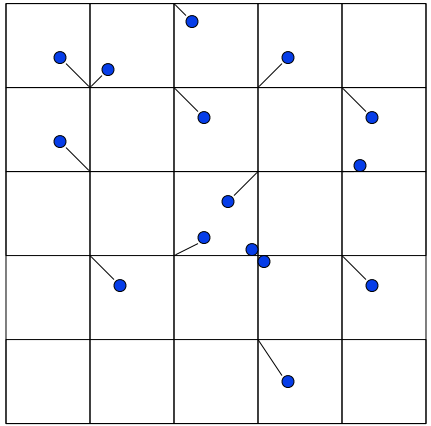
- Exploit fact that the far-field force satisfies a partial differential equation (PDE) that is easy to solve on a regular mesh:
  - FFT, multigrid *(future lectures)*
  - **Cost is reduced to O(n log n) or O(n)** from $O(n^2)$

- The accuracy depends on the fineness of the grid is and the uniformity of the particles

Far-Field Forces: Particle-Mesh Method

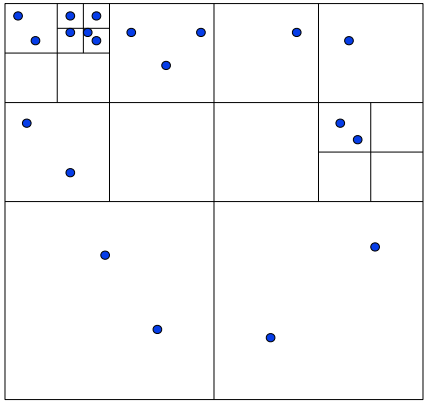
- Based on approximation
  - It superimposes a (mathematical) regular mesh
  - And, **“move”** particles to nearest grid point

1. Particles are moved to nearby mesh points (**scatter**)
2. Solve mesh problem
3. Forces are interpolated at particles from mesh points (**gather**)



Far-Field Forces: Tree Decomposition

- Based on approximation
  - Forces from group of far-away particles “simplified” — resembles a single large particle
  - Use tree; each node contains an approximation of descendants
- **O(n log n) or O(n)** instead of $O(n^2)$
- Implemented in several algorithms:
  - **Barnes-Hut (early 1980s)**
  - **Fast multipole method (FMM)** of Greengard/Rohklin
  - Anderson’s method



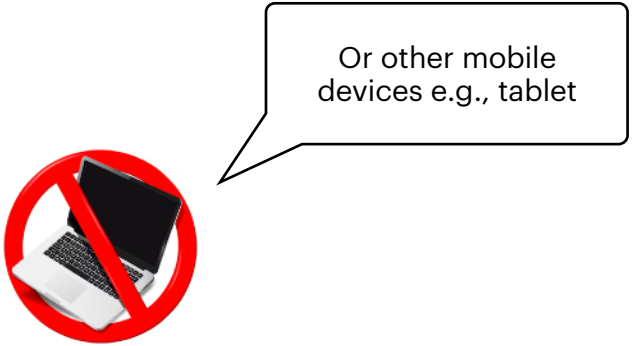
Particles Systems Summary

- The model contains discrete entities, namely, particles
- The simulation follows particles through time steps:
  - The force is composed of external force, nearby force, and far field force
  - A classic all-pairs algorithm is simple, but inefficient, $O(n^2)$
  - **Particle-mesh methods** approximates by moving particles to a regular mesh, where it is easier to compute forces
  - **Tree-based algorithms** approximate by treating set of particles as a group, when far away

Plan for Today

- Locality and Simulation: Particles Simulation
- Locality and Simulation: ODEs and PDEs **(aka SpMV)**

no smartphones



no laptop

Why? Research shows that they slow us down in a learning environment.
I recommend **paper and pen** for taking notes

Credit to: https://users.cs.utah.edu/~pandey/courses/cs6530/fall23/index.html

Basic Types of Simulation

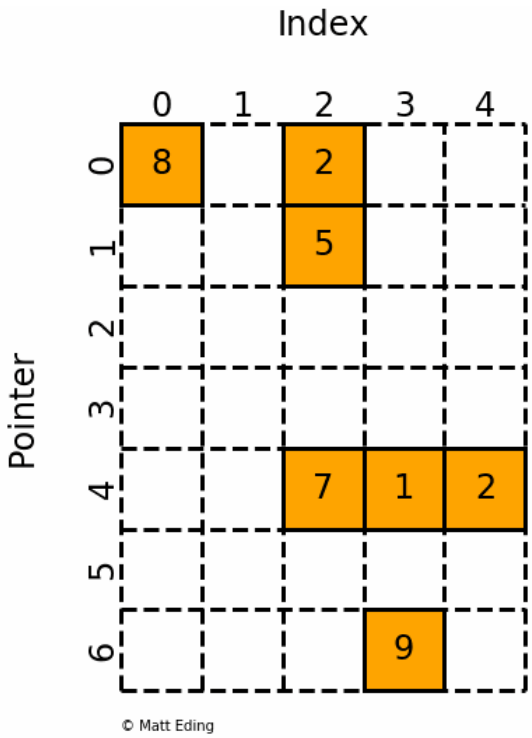
- Discrete event systems
  - *"Game of Life," Manufacturing systems, Finance, Circuits, Pac-man, etc*
- Particle systems:
  - *Billiard balls, Galaxies, Atoms, Circuits, Pinball, etc.*
- Lumped variables depending on continuous parameters
  - aka Ordinary Differential Equations (ODEs)
  - *Structural mechanics, Chemical kinetics, Circuits*
- Continuous variables depending on continuous parameters
  - aka Partial Differential Equations (PDEs)
  - *Heat, Elasticity, Electrostatics, Finance, Circuits, Medical Image Analysis*

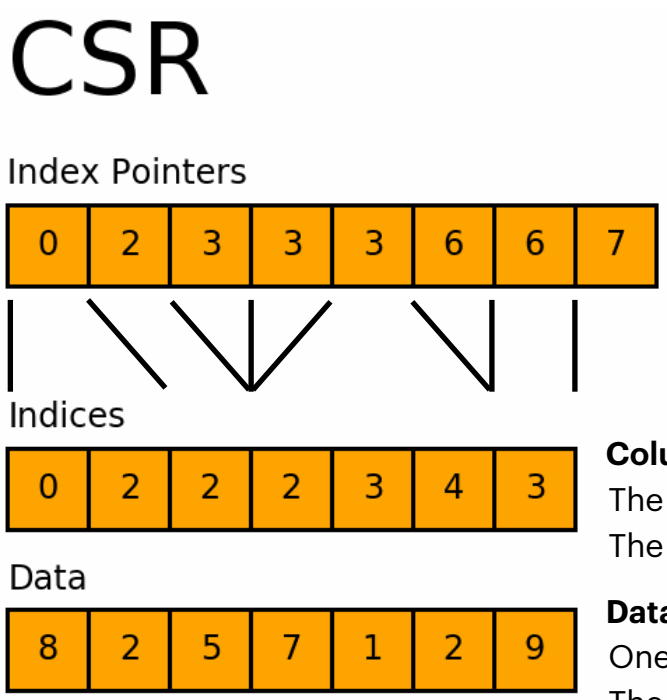
Sparse Matrix Vector Multiply (SpMV)

To review at home, not covered in class in Spring 2026

Sparse Matrix Vector Multiply (SpMV)

Compressed Sparse Row (CSR)





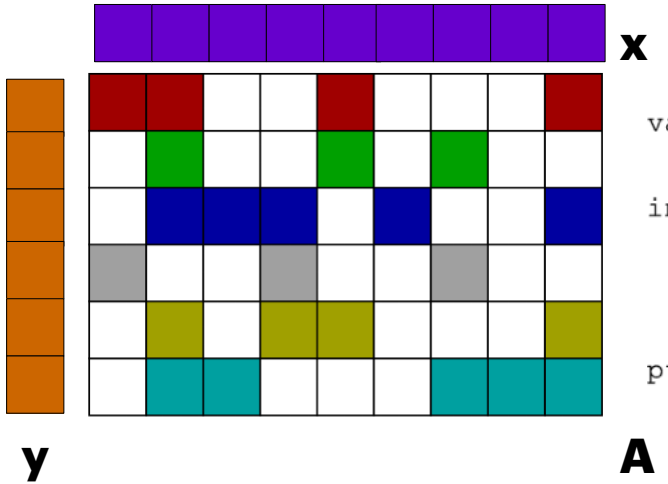
Row pointer
The **index** is the row id
The **values** are [start:stop] in the column indices array

Column indices
The **length** is the number of nonzeros
The **values** are column indices

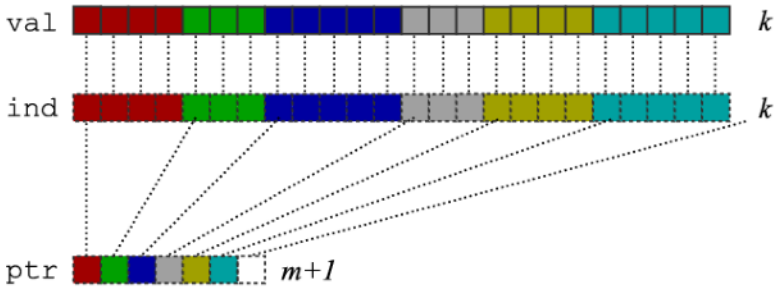
Data values
One-to-one correspondence with column
The **values** are the nonzeros

SpMV

SpMV kernel: y(i) = y(i) + A(i, j)·x(j)



Representation of **A**



for each row **i**

for **k** = **ptr[i]** to **ptr[i+1]-1** do

y[i] = **y[i]** + **val[k]** * **x[ind[k]]**

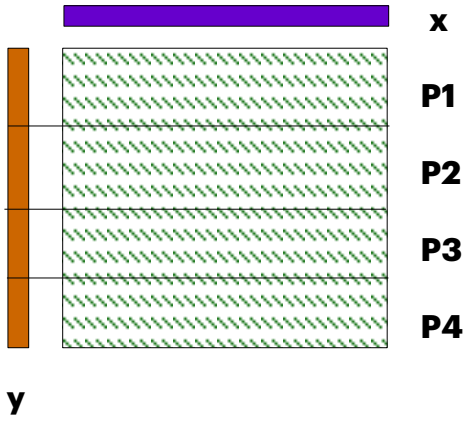
Parallel SpMV

SpMV kernel: $y(i) = y(i) + A(i, j) \cdot x(j)$

Questions:

- On which processors we store $y(i)$, $x(i)$, and $A(i, j)$?
- On which processors we compute $y(i)$= sum (from 1 to n) $A(i, j) * x(j)$ = (row i of A) * x?

A sparse dot product!



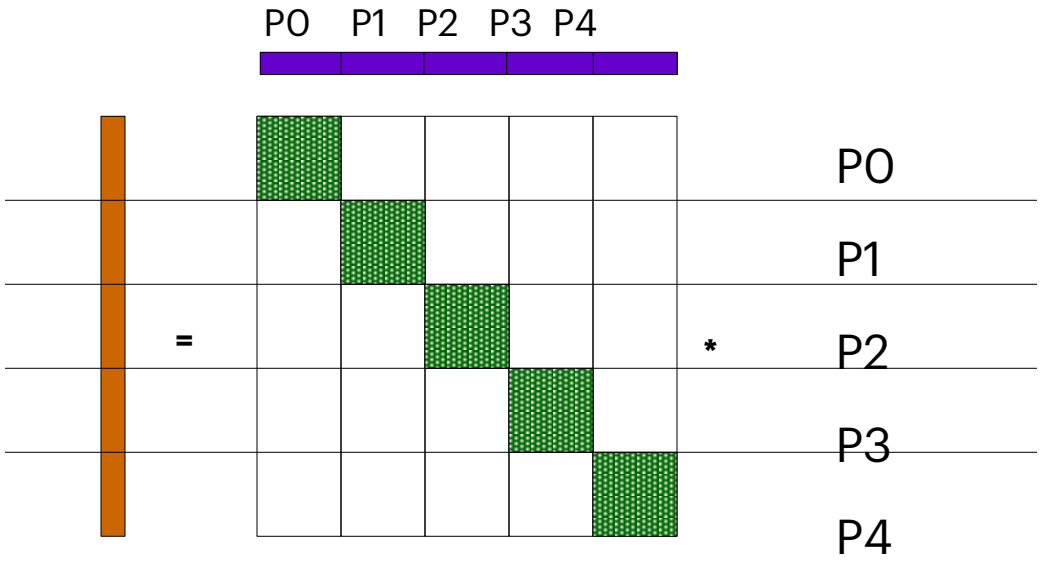
Partitioning:

- Partition index set $\{1, \ldots, n\} = N_1 \cup N_2 \cup \ldots \cup N_p$
- For all i in N_k, processor k stores y[i], x[i], and row i of A
- For all i in N_k, processor k computes y[i] = (row i of A) * x

"Owner computes" rule: processor k computes the y[i]s it owns!

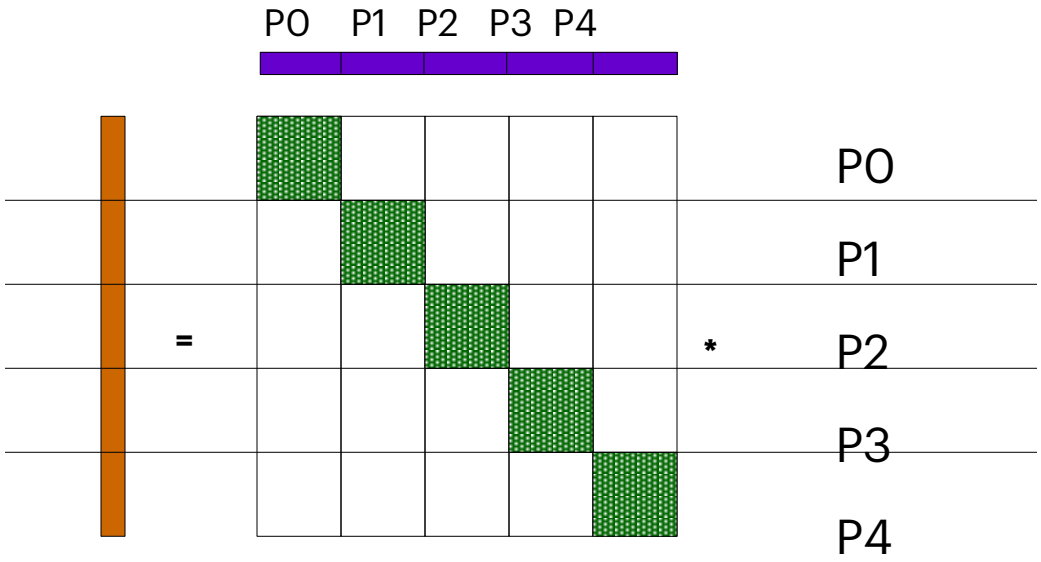
Sparse Matrix Reordering

- The ideal matrix structure for parallelism is **block diagonal**



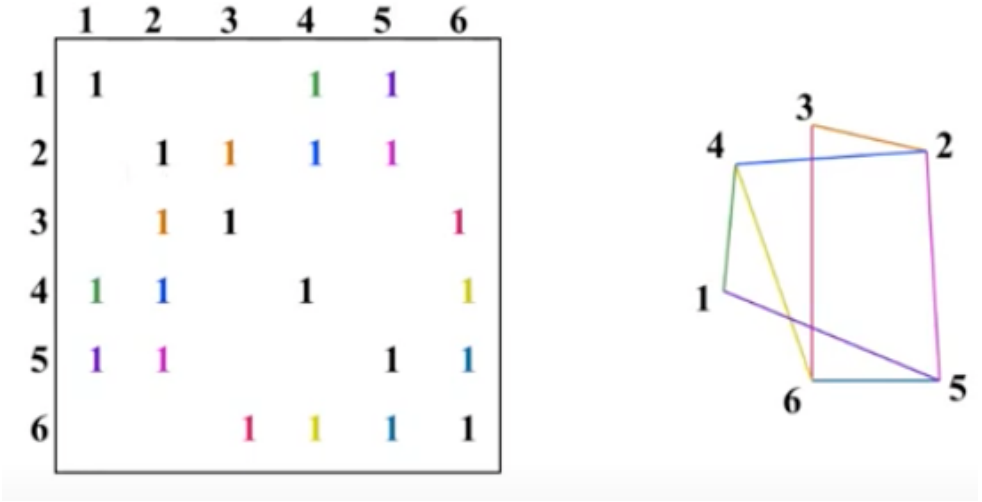
Sparse Matrix Reordering

- The ideal matrix structure for parallelism is **block diagonal**
- Can we reorder the rows/columns to get close to this?



Graph Partitioning

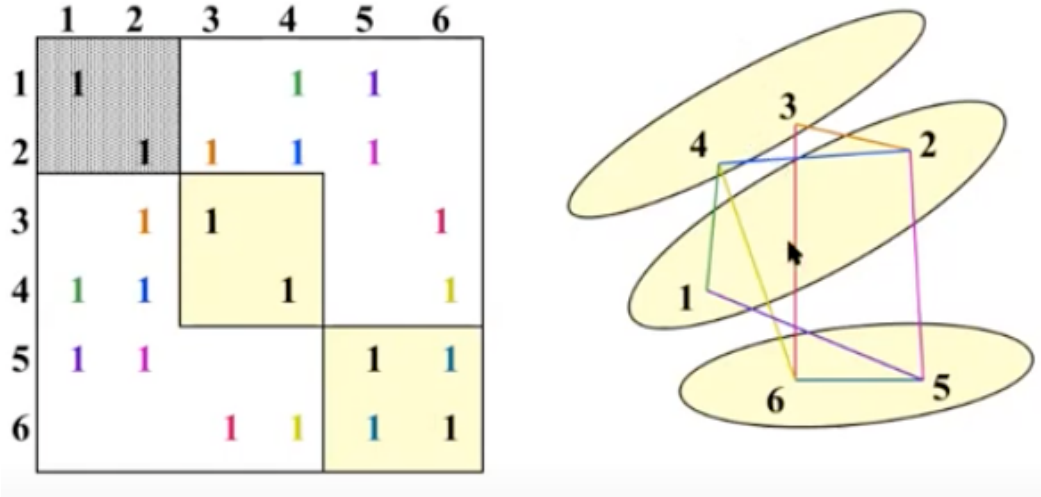
Relationship between matrix and graph:



- A edge in the graph is a nonzero in the matrix
- If divided over 3 processors, there are 14 nonzeros outside the diagonal blocks, which are the 7 (bidirectional) edges

Graph Partitioning

Relationship between matrix and graph:



- A edge in the graph is a nonzero in the matrix
- If divided over 3 processors, there are 14 nonzeros outside the diagonal blocks, which are the 7 (bidirectional) edges

Common Problems Summary

- Load balancing:
  - Done **statically**, using graph partitioning
    - Discrete event simulation
    - SpMV
  - Done **dynamically** if workload changes significantly at runtime
- Linear algebra:
  - Require to solve linear systems (sparse and dense)
  - Eigenvalues problems use similar techniques
- Fast particle methodologies:
  - $O(n \log n)$ instead of $O(n^2)$

Lumped Systems: Ordinary Differential Equation (ODEs)

Systems of Lumped Variables

In a *lumped system*, **the dependent variables of interest are a function of time alone**, i.e., need to solve a set of ordinary differential equations (ODEs)

- E.g., circuit:
  - It's approximated as graph, which in turn is a **sparse matrix**
  - The wires are edges
  - Each node is a connection between 2 or more wires
    - Each edge has resistor, capacitor, inductor or voltage source
    - The system is "lumped" because we are not computing the voltage/current at every point in space along a wire, **just at the endpoint**
  - The variables are related by Ohm's Law, Kirchoff's Laws, etc.
- ODEs are **differentiated with respect to time**

Circuit Example

- The state of the system is represented by:
  - $v_n(t)$ node voltages
  - $i_b(t)$ branch currents any time t
  - $v_b(t)$ branch voltages

- Equations include:
  - Kirchoff's current : $A^*i_b = 0$
  - Kirchoff's voltage : $A'^*v_n - v_b = S$
  - Ohm's law
  - Capacitance
  - Inductance

$$\begin{pmatrix} 0 & A & 0 \\ A' & 0 & -I \\ 0 & R & -I \\ 0 & -I & C*d/dt \\ 0 & L*d/dt & I \end{pmatrix} * \begin{pmatrix} v_n \\ i_b \\ v_b \end{pmatrix} = \begin{pmatrix} 0 \\ S \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

- A is a large **sparse matrix**, representing connections in circuit:
  - One column per branch (edge), one row per node with +1 and -1 in each column at any row indicating an end point

How to Solve ODEs

In most cases, the matrices are **sparse**:

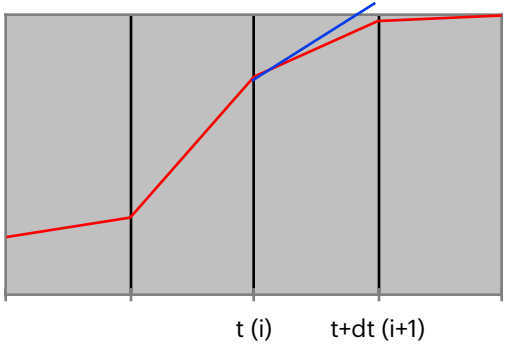
- i.e., most array entries are 0
- You don't store and don't compute on these 0's
- It's sparse because each component depends on only a few others

- Given a set of ODEs, two kinds of questions are:
  - Compute the values of the variables at some time *t*
    - **Explicit** methods
    - **Implicit** methods
  - Compute modes of vibration
    - Eigenvalue problems

How to Solve ODEs Explicitly

Let us assume ODE is x'(t) = f(x) = A*x(t), where **A** is a **sparse matrix**

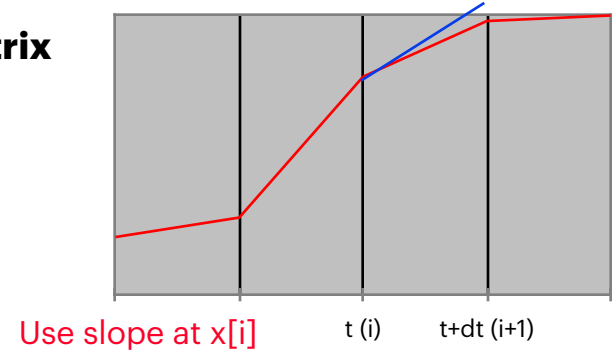
- Compute x(i*dt) = x[i] at i=0, 1, 2, ... (x[i] is an array)
- ODE gives x'(i*dt) = slope, then x[i+1]=x[i] + dt*slope



How to Solve ODEs Explicitly

Let us assume ODE is x'(t) = f(x) = A*x(t), where **A** is a **sparse matrix**

- Compute x(i*dt) = x[i] at i=0, 1, 2, ... (x[i] is an array)
- ODE gives x'(i*dt) = slope, then x[i+1]=x[i] + dt*slope



Explicit methods, e.g., (Forward) Euler's method:

- Let approximate x'(t)=A*x(t) by (x[i+1] - x[i])/dt = A*x[i]
- x[i+1] = x[i]+dt*A*x[i], i.e. **sparse matrix-vector multiplication.** Then, compute x(i*dt) = x[i] at i = 0, 1, 2, ...

Tradeoffs:

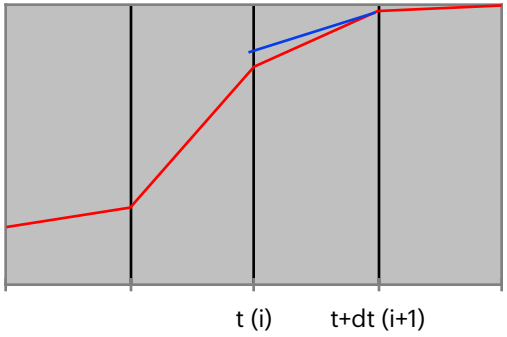
- It's simple algorithm: sparse matrix vector multiply
- But has stability problems: may need to take very small time steps, especially if system is **stiff** (i.e. A has some large entries, so x can change rapidly)

How can I take a larger time step even if the solution is quite steep?

How to Solve ODEs Implicitly

Let us assume ODE is x'(t) = f(x) = A*x(t), where **A** is a **sparse matrix**

- Compute x(i*dt) = x[i] at i=0, 1, 2, ... (x[i] is an array)
- ODE gives x'(i*dt) = slope, then x[i+1]=x[i] + dt*slope

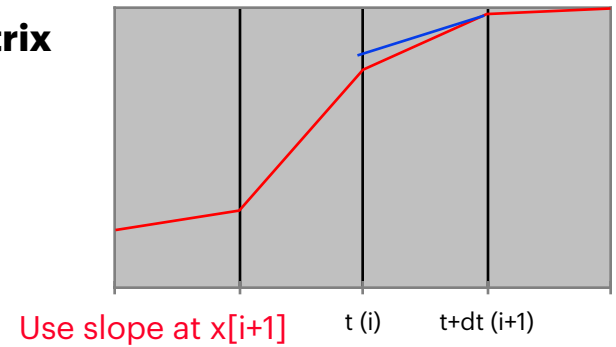


I can take the derivate at time t(i+1) instead of t(i)

How to Solve ODEs Implicitly

Let us assume ODE is x'(t) = f(x) = A*x(t), where **A** is a **sparse matrix**

- Compute x(i*dt) = x[i] at i=0, 1, 2, ...
- ODE gives x'((i+1)*dt) = slope, then x[i+1]=x[i] + dt*slope



Implicit method, e.g., Backward Euler solve:

- Let approximate x'(t)=A*x(t) by (x[i+1] - x[i])/dt = A*x[i+1]
- x[i] = (I - dt*A)*x[i+1], i.e. we need to solve a **sparse linear system of equations**

Tradeoffs:

- Larger timestep possible: especially for **stiff** problems
- But it's a **more difficult algorithm**: need to solve a sparse linear system of equations at each step

Eigensolver

Computing modes of vibration, finding eigenvalues and eigenvector:

- You seek solution of $d^2x(t)/dt^2 = A*x(t)$ of form $x(t) = \sin(\omega*t) * x_0$, where x_0 is a constant vector
- ω called the frequency of vibration
- x_0 sometimes called a "mode shape"
- Plug in to get $-\omega^2 * x_0 = A* x_0$, so that $-\omega^2$ is an **eigenvalue** and x_0 is an **eigenvector** of A

The **solution schemes** is either **sparse-matrix multiplication** or solving **sparse linear systems**

ODEs Methods Summary

- **Explicit** methods for ODEs need sparse-matrix-vector multiplication
- **Implicit** methods for ODEs need to solve linear systems

- Conclusion: **SpMV is key to many ODE problems!**
  - Relatively simple algorithm to study in detail
  - Two key problems: locality and load balance

ODEs Methods Summary

- **Explicit** methods for ODEs need sparse-matrix-vector multiplication
- **Implicit** methods for ODEs need to solve linear systems